

Práctica Final: Sistema Multiagente Financiero con LangGraph, FastAPI y Observabilidad Humana



Autores:

Diego Esclarín (51729611N)
Sofía Contreras (09848471D)

Aprendizaje en Inteligencia Artificial
Grado en Ingeniería en Inteligencia Artificial
Universidad Rey Juan Carlos
Curso 2025–26

Índice

1. Introducción	7
2. Contexto	9
2.1. Integración de P1–P5	9
2.2. Requisitos funcionales	9
2.3. Restricciones técnicas y de despliegue	10
3. Estado del Arte	11
3.1. Orquestación de agentes con LangGraph	11
3.2. LLM y <i>providers</i>	11
3.3. Visión por computador: OCR y biometría	11
3.4. Detección de anomalías y predicción temporal	11
3.5. Clasificación de texto financiero	12
3.6. Observabilidad: Langfuse	12
3.7. Prompt engineering: ASPECCT vs TAREA	12
3.8. Pruebas de carga: Locust	12
4. Propuesta	13
4.1. Arquitectura general	13
4.2. Roles diferenciados (usuario vs. administrador)	14
4.3. Módulos P1–P5 como microservicios REST	15
4.4. Memoria conversacional y recuperación de sesiones	16
4.5. Diálogo natural con detección de intención	16
4.6. Diagramas visuales dinámicos (XAI)	17
4.7. Monitorización automática: MonitorAgent	18
4.8. Observabilidad: Langfuse	19
4.9. Prompt engineering con ASPECCT	20
4.10. Evoluciones incrementales sobre P2, P3 y P5	20
4.11. Pruebas unitarias y de estrés	24
5. Conclusiones	25

Índice de figuras

1.	Topología general del sistema multiagente. El Orquestador es el único nodo con acceso al LLM y delega en tres sub-agentes deterministas que persisten en Supabase.	14
2.	Flujo interno del Orquestador. En función de si el estado contiene datos de sub-agente, el LLM opera en modo <code>_route</code> (<i>structured output</i>) o <code>_narrate</code> (texto plano). El bucle se acota a seis iteraciones para evitar delegaciones infinitas.	14
3.	Pantalla de <i>login</i> de usuario regular. La captura de webcam (<code>getUserMedia</code> + <code>MediaRecorder</code>) produce un vídeo WebM de 3,5s que se envía como <code>face_video</code> al pipeline biométrico de Security (MTCNN + FaceNet + DenseNet201).	15
4.	Captura del chat del usuario. Cada respuesta del Orquestador incluye texto narrado y, cuando procede, una visualización dinámica generada a partir del <code>ChatResponse.chart</code>	17
5.	Pantalla de revisión de transacciones pendientes. Los movimientos detectados como anómalos por el agente Security se persisten con <code>status='pending'</code> y se muestran al usuario para confirmación o rechazo manual.	18
6.	Latencias p50 y p95 por agente extraídas en vivo de la tabla <code>events</code> del <code>MonitorAgent</code> . Datos: <code>logs/app.log</code> , solo eventos con <code>status="ok"</code> . Se omite <code>admin_orchestrator</code> (n=7) por estar dominado por un arranque en frío del LLM.	19
7.	Accuracy <i>cold-start</i> por clase, baseline (P2 original) frente a post-E1 (transformer multilingüe + zero-shot). Datos: <code>doc/results/p2_baseline.json</code> y <code>p2_post.json</code>	21
8.	Brecha de idioma ES vs EN antes y después de E1. El modelo multilingüe cierra completamente el sesgo del char-ngrams monolingüe e iguala (incluso supera ligeramente) la accuracy en inglés.	22
9.	Ejemplo de factura en euros utilizada para la evaluación del motor OCR enriquecido (E2). El reto consiste en extraer <code>total</code> , <code>fecha</code> , <code>NIF</code> , <code>comercio</code> e <code>IVA</code> de imágenes con calidad variable.	22
10.	Resultados pre/post de E2 sobre el <i>set</i> EUR de 30 facturas sintéticas. El IVA queda al 63 % por la variabilidad de formatos (IVA 21%: vs I.V.A.:), el resto de campos llega al 100 %. Datos: <code>doc/results/p3_eur_baseline.json</code> y <code>p3_eur_post.json</code>	23
11.	Ejemplos de muestras reales (<i>live</i>) frente a ataques (<i>spoof</i>) usados en la evaluación de <i>liveness</i> . El <i>anti-spoofing</i> implementado en E3 extiende esta detección con LBP, análisis de espacio de color y FFT, sin requerir un modelo nuevo.	23

Índice de cuadros

1.	Mapeo entre los requisitos del enunciado y las secciones de la propuesta. . .	9
2.	Métricas pre/post de la evolución E1 (clasificador semántico multilingüe para la Area de P2).	21
3.	Métricas pre/post de la evolución E2 (OCR de facturas en euros con campos enriquecidos y InvoiceMetadata).	22
4.	Estado de las fases y métricas verificadas de la evolución E3 (biometría con vídeo y anti-spoofing escalonado de P5).	24
5.	Distribución de los <i>tests</i> unitarios y de integración del proyecto.	24

1. Introducción

La memoria documenta el desarrollo de la práctica final de la asignatura *Aprendizaje en Inteligencia Artificial* del Grado en Ingeniería en Inteligencia Artificial de la Universidad Rey Juan Carlos. El proyecto, denominado internamente **PFINAL _AP-IA**, constituye la finalización e integración de las seis prácticas anteriores (P1–P5) bajo un único sistema multiagente conversacional dedicado a la gestión financiera personal.

El sistema toma la forma de un asistente financiero capaz de mantener un diálogo natural con el usuario, registrar transacciones a partir de texto o imágenes (OCR), detectar anomalías de gasto, generar predicciones temporales, autenticar mediante biometría facial y explicar sus respuestas mediante visualizaciones dinámicas. La arquitectura se apoya en **LangGraph** [1] para la orquestación de agentes, **FastAPI** [2] para exponer cada módulo como microservicio REST, **Next.js** [3] en el frontend y **Langfuse** [4] para la monitorización de trazas, llamadas a herramientas, latencias y consumo de tokens.

El documento se estructura en cinco capítulos. El capítulo de *Contexto* sitúa la práctica dentro del recorrido de las prácticas anteriores y recoge los requisitos funcionales. El *Estado del arte* repasa los frameworks y técnicas que sustentan las decisiones de diseño. La *Propuesta* detalla la arquitectura implementada, los dos roles diferenciados (usuario y administrador), la memoria conversacional, la distribución de los módulos P1–P5 como microservicios, la evolución incremental sobre P2, P3 y P5, las metodologías de prompt engineering empleadas y la estrategia de pruebas unitarias y de estrés. Las *Conclusiones* resumen los resultados cuantitativos obtenidos y las líneas de trabajo futuro.

2. Contexto

2.1. Integración de P1–P5

A lo largo de la asignatura se han implementado cinco prácticas independientes pero complementarias, cada una orientada a un eje distinto del aprendizaje automático aplicado al dominio financiero personal:

- **P1 — Predicción temporal.** Modelos de regresión y series temporales (Random Forest, HistGradientBoosting y ARIMA) para anticipar el gasto del mes siguiente.
- **P2 — Clasificación multietiqueta.** Asignación de la categoría (*Area*) a una transacción a partir de su descripción textual mediante un `SGDClassifier` sobre char *n*-gramas.
- **P3 — OCR financiero.** Extracción del importe total de facturas reales mediante PaddleOCR y un scorer basado en Gradient Boosting.
- **P4 — Agente conversacional.** Primer contacto con LangGraph y la orquestación de herramientas mediante LLM.
- **P5 — Biometría facial y detección de anomalías.** Pipeline MTCNN + FaceNet + DenseNet201 para autenticación biométrica con *liveness*, e `IsolationForest` para anomalías financieras.

La práctica final exige evolucionar la práctica 6, que integra las cinco anteriores bajo un único sistema multiagente que conserva la fiabilidad de las soluciones individuales ganando cohesión, persistencia multiusuario y observabilidad profesional. El enunciado considera P4 cubierto, por lo que la *evolución incremental* (Sección 4.10) recae sobre P2, P3 y P5.

2.2. Requisitos funcionales

La especificación de la práctica final incluye un conjunto de requisitos transversales que el sistema debe satisfacer. La Tabla 1 los enumera y los enlaza con la sección de la propuesta en la que se justifica su cumplimiento.

Requisito	Sección de la propuesta
Histórico conversacional para el LLM	§4.4
Recuperación de chats previos	§4.4
Diagramas visuales dinámicos (XAI)	§4.6
Monitorización del sistema (MonitorAgent)	§4.7
Al menos dos roles diferenciados en LangGraph	§4.2
Módulos P1–P5 distribuidos vía API REST	§4.3
Diálogo natural con detección de intención	§4.5
Uso de Langfuse	§4.8
Prompt engineering (ASPECCT)	§4.9
Tres evoluciones incrementales (P2, P3, P5)	§4.10
Evaluaciones cuantitativas	§4.10
Pruebas unitarias y de estrés	§4.11

Cuadro 1: Mapeo entre los requisitos del enunciado y las secciones de la propuesta.

2.3. Restricciones técnicas y de despliegue

El sistema se ha diseñado con tres restricciones operativas relevantes que han condicionado tanto la elección de tecnologías como la arquitectura final.

Stack gratuito o de coste mínimo. El LLM principal corre en *Groq Cloud* (plan gratuito con *rate limits* de 30 *requests*/min para *llama-3.3-70b-versatile*), la base de datos vive en *Supabase* (Postgres gestionado con 500 MB en el plan *free*), la observabilidad en *Langfuse Cloud* (50.000 *traces*/mes gratuitos) y el frontend en *Vercel*. El *backend* se sirve localmente con *uvicorn* y se expone públicamente vía un túnel *ngrok* con dominio estático, ya que el plan gratuito de Vercel no permite contenedores con dependencias nativas pesadas (PaddlePaddle, PyTorch, OpenCV) en sus *serverless functions*.

Compatibilidad de versiones. PaddleOCR 2.6.2 exige *numpy*<2.0, lo que ancla todo el *stack* a NumPy 1.26.4. Esta restricción ha condicionado la elección del resto de dependencias: *scikit-learn* 1.5.0, *torch* 2.5.1, *langgraph* 1.x, *facenet-pytorch* (en lugar de *deepface*, que arrastra TensorFlow). El *trade-off* aceptado es no poder usar *numpy* 2.x ni las versiones más recientes de *scikit-learn*; a cambio, todo el *stack* convive en un único *entry point* (*uvicorn*) sin necesidad de aislar P3 en un microservicio aparte.

Activos pesados con Git LFS. Los modelos preentrenados (*models/area_classifier.joblib*, *models/ocr_total_extractor.joblib* y *models/liveness_kaggle.pth*, ~78 MB en total) viajan por *Git Large File Storage* para no inflar el historial del repositorio. Sin habilitar *git lfs install* antes del *clone*, los modelos se descargan como punteros de 134 bytes y el *backend* falla al cargar el clasificador de Area en el primer turno. Adicionalmente, las dependencias DL (PaddleOCR, PyTorch, MediaPipe) ocupan ~2,5 GB en *.venv*, lo que descarta la idea inicial de empaquetar la aplicación en una imagen Docker desplegable en plataformas *serverless* sin almacenamiento persistente.

Cumplimiento RGPD y aislamiento multiusuario. El sistema maneja datos personales sensibles (transacciones financieras y *embeddings* biométricos), por lo que el cumplimiento del Reglamento General de Protección de Datos no es una característica opcional sino una restricción de diseño. Esto se traduce en cuatro exigencias concretas: (i) los *embeddings* faciales se almacenan cifrados en disco con **Fernet** (AES-128) sobre una clave derivada con **PBKDF2-HMAC-SHA256** a 310.000 iteraciones, nunca en claro; (ii) las contraseñas se almacenan con **bcrypt** (*cost factor* 12) sin recurrir a **passlib** — incompatible con **bcrypt** 4.x; (iii) las notificaciones a Telegram nunca exponen importes ni descripciones salvo que el usuario active `notification_level='full'`; y (iv) la detección de anomalías nunca compara las transacciones de un usuario contra el histórico de otro.

3. Estado del Arte

Esta sección revisa los marcos teóricos y las herramientas en las que se apoya el sistema, distinguiendo aquellas que han sido finalmente adoptadas de las alternativas evaluadas y descartadas.

3.1. Orquestación de agentes con LangGraph

LangGraph [1] (LangChain Labs) es una biblioteca para construir sistemas multiagente como grafos dirigidos cuyos nodos son funciones Python y cuyas aristas pueden ser condicionales, ejecutadas en función del estado tipado del grafo. Frente a alternativas como AutoGen (Microsoft) o CrewAI, LangGraph destaca por:

- Su **enfoque grafo + estado tipado**, que permite razonar formalmente sobre la trayectoria del control y garantiza la *terminación* (en este proyecto se impone un *iteration limit* de seis pasos).
- Su **soporte nativo de checkpoints** (MemorySaver, PostgresSaver, SqliteSaver), que guarda el histórico conversacional dentro del mismo grafo sin necesidad de un sistema externo.
- La posibilidad de combinar *structured output* (Pydantic) con LLM *plain* en distintos nodos del mismo grafo. En el sistema se aprovecha para separar *routing* (decisión) y *narración* (texto natural).

3.2. LLM y providers

El catálogo de proveedores ha quedado configurable por usuario, con soporte nativo para Groq [5], OpenAI, Anthropic y Google. El modelo por defecto es llama-3.3-70b-versatile servido por Groq, que ofrece latencias inferiores a 500 ms en *tool calling* gracias a su hardware LPU dedicado. Para tareas de *small-talk* y resumen rodante se usa llama-3.1-8b-instant, que reduce el gasto de tokens en operaciones poco exigentes.

3.3. Visión por computador: OCR y biometría

Para OCR se adoptó PaddleOCR [6] (versión 2.6.2) por su rendimiento sobre texto en castellano y su capacidad de funcionar en CPU sin GPU externa, descartando EasyOCR (menor precisión sobre tickets en euros). Para biometría se eligió facenet-pytorch [7] (InceptionResnetV1 pre-entrenado en VGGFace2) en lugar de deepface/ArcFace, evitando así introducir TensorFlow en el stack. La detección de rostros se apoya en MTCNN [8] y la detección de vida (*liveness*) en DenseNet201 [9] con pesos ImageNet, completada con un módulo *anti-spoofing* basado en LBP, análisis de espacio de color HSV/YCrCb y análisis frecuencial (FFT) para detectar patrones Moiré de pantallas LCD. Los *landmarks* faciales necesarios para el *challenge-response* de parpadeo se extraen con MediaPipe Face Mesh [10].

3.4. Detección de anomalías y predicción temporal

El detector de anomalías financieras combina IsolationForest [11] de scikit-learn con una regla heurística de 3-sigma sobre los importes históricos del

usuario. Esta combinación, ya validada en P5, se ha mantenido por su robustez y por la interpretabilidad de la regla 3-sigma frente al usuario final. Para la predicción mensual conviven tres modelos — Random Forest, HistGradientBoosting y ARIMA — accesibles vía un único endpoint REST que permite seleccionar el método.

3.5. Clasificación de texto financiero

Para clasificar la *Area* de una transacción se ha evolucionado del `SGDClassifier` sobre TF-IDF de char n -gramas (P2 original) hacia un pipeline híbrido basado en *Sentence-BERT* [12] con embeddings multilingües. Este enfoque permite generalizar a descripciones que no aparecieron en el entrenamiento (*cold-start* y nuevos idiomas) sin perder el *macro-F1* en el conjunto de entrenamiento.

3.6. Observabilidad: Langfuse

`Langfuse` [4] (versión 3) es una plataforma de observabilidad específica para LLM con instrumentación nativa de LangChain/LangGraph mediante `CallbackHandler`. Recoge automáticamente la cadena de llamadas (*traces*), prompt completo, respuesta cruda, *tool calls*, latencias y consumo de tokens. Frente a alternativas como `LangSmith` (de pago), `Helicone` (no LangChain-native) o `Phoenix` (Arize), Langfuse ofrece un plan gratuito generoso y un *dashboard* web suficiente para auditar la ejecución de los agentes en tiempo real.

3.7. Prompt engineering: ASPECCT vs TAREA

El enunciado permite escoger entre dos metodologías estructuradas de *prompting*. ASPECCT [13] (Audience, Style, Purpose, Examples, Constraints, Context, Tone) separa *style* y *tone*, lo que en el sistema permite inyectar el bloque de estilo dependiente del rol del usuario (`basic/advanced`) como un `SystemMessage` independiente sin tocar el prompt principal. TAREA (Tarea, Audiencia, Recursos, Estructura, Activación) resulta más compacta pero no separa estilo y tono. Por esta razón se ha adoptado ASPECCT como metodología oficial del proyecto.

3.8. Pruebas de carga: Locust

Para validar el comportamiento del sistema bajo concurrencia se ha optado por `Locust` [14], un *framework* de *load testing* en Python que permite describir el comportamiento de los usuarios virtuales como clases con `@task`. La principal ventaja sobre alternativas como JMeter o k6 es que los escenarios se escriben en el mismo lenguaje del backend, lo que facilita la generación de cargas realistas (registro con *multipart*, JWT, etc.) sin duplicar lógica.

4. Propuesta

Esta sección describe la solución implementada, organizada en torno a los doce requisitos del enunciado.

4.1. Arquitectura general

El sistema sigue una arquitectura en tres capas. La capa de **presentación** la conforma un frontend en Next.js 14 (App Router) desplegado en Vercel, con dos dominios funcionales separados: `/chat` (financiero, usuario regular) y `/admin/chat` (operativo, administrador). La capa de **aplicación** es una API FastAPI servida por Uvicorn, que expone tanto los endpoints de chat como los microservicios de los módulos `/modules/p1` hasta `/modules/p5`. La capa de **datos** la forman una base de datos PostgreSQL alojada en Supabase [15] y un *store* cifrado en disco para los *embeddings* biométricos. La Figura 1 resume esta topología.

El núcleo lógico es un grafo LangGraph [1] cuyo nodo raíz es el **Orquestador**. El Orquestador es el único componente con acceso al LLM; los sub-agentes (Analyst, Registrar, Security, Conversational) son funciones deterministas que devuelven *dataclasses* Pydantic. El grafo está acotado a seis iteraciones máximas para evitar bucles, y emplea `JsonPlusSerializer` con tipos registrados explícitamente para silenciar los *warnings* de deserialización de LangGraph 1.x.

Reglas de oro del diseño. La arquitectura se rige por seis invariantes que se respetan en todo el código:

1. **Solo el Orquestador habla con el usuario.** Los sub-agentes nunca devuelven texto libre, solo *dataclasses* firmadas (`AnalysisReport`, `RegistryResult`, `SecurityVerdict`). El texto final lo genera el narrador del Orquestador.
2. **Solo el Orquestador usa LLM.** Security, Registrar y Analyst son deterministas: modelos clásicos (`IsolationForest`, `SGDClassifier`, `PaddleOCR`) más reglas. Esto facilita los *tests* y garantiza reproducibilidad.
3. **Aislamiento de estado.** Cada sub-agente escribe en su *slot* del `OrchestratorState` (`analysis_report`, `registry_result`, `security_verdict`), sin interferir entre sí.
4. **Bucle controlado.** El flujo *routing* $\rightarrow$ sub-agente $\rightarrow$ narración $\rightarrow$ END lleva un cinturón anti-bucles de seis iteraciones máximas.
5. **Notificaciones disparadas en el agente de origen,** nunca desde el *frontend*: si Security marca una transacción como anómala, el aviso a Telegram se dispara en el propio agente, en el mismo turno.
6. **Los datos del usuario solo se validan contra su propio histórico.** Security usa `load_user_history_db_only`, nunca cae al CSV de demostración. Esta regla nació de un *bug* en el que las anomalías se evaluaban contra el histórico de prueba.

La Figura 2 ilustra el doble modo *route/narrate* del Orquestador y el bucle controlado con los sub-agentes. La detección de modo se reduce a comprobar si el estado ya contiene datos de algún sub-agente (`_has_subagent_data(state)`): si no los hay, el LLM se invoca con *structured output* para producir una `OrchestratorDecision`; si los hay, se invoca en modo plano para narrar el resultado en español adaptado al rol del usuario.

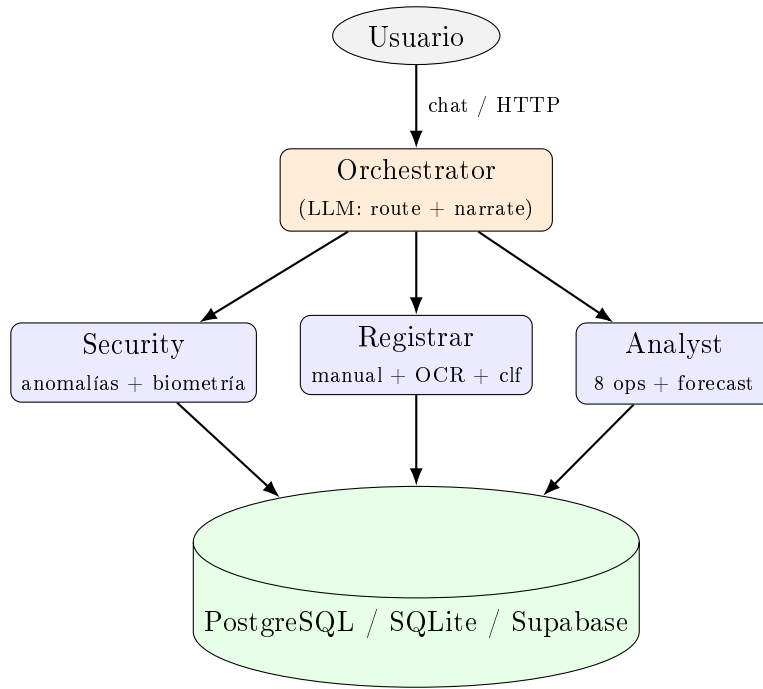


Figura 1: Topología general del sistema multiagente. El Orquestador es el único nodo con acceso al LLM y delega en tres sub-agentes deterministas que persisten en Supabase.

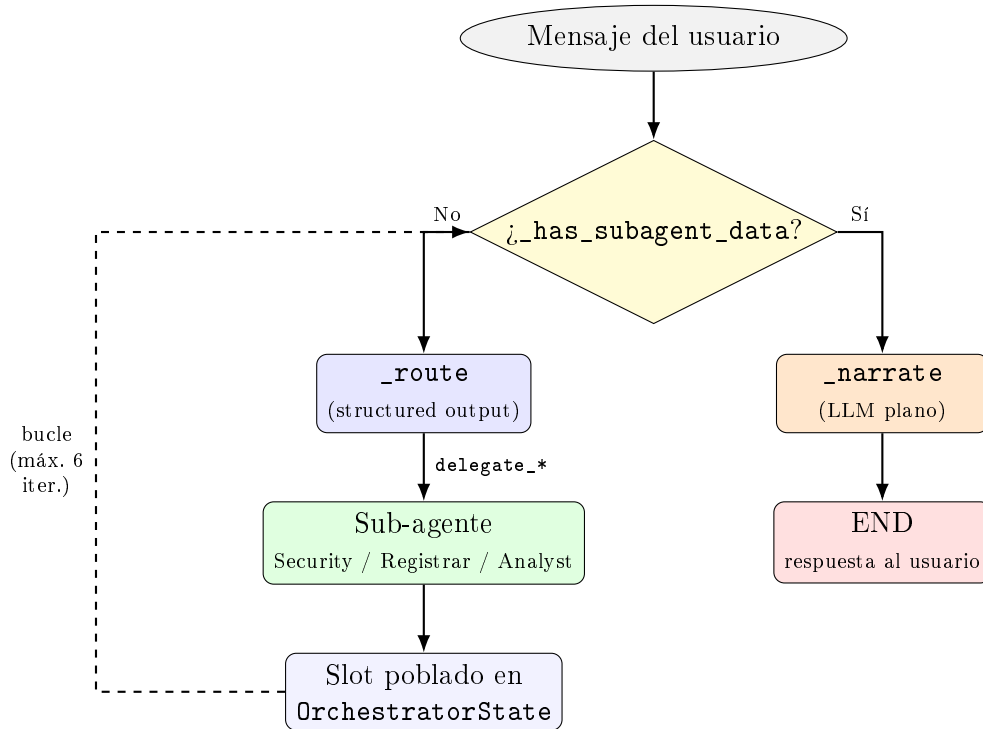


Figura 2: Flujo interno del Orquestador. En función de si el estado contiene datos de sub-agente, el LLM opera en modo `_route` (*structured output*) o `_narrate` (texto plano). El bucle se acota a seis iteraciones para evitar delegaciones infinitas.

4.2. Roles diferenciados (usuario vs. administrador)

Cumple el requisito de al menos dos roles en LangGraph. El sistema implementa dos grafos separados, cada uno con su propio LLM, su conjunto de *tools* y

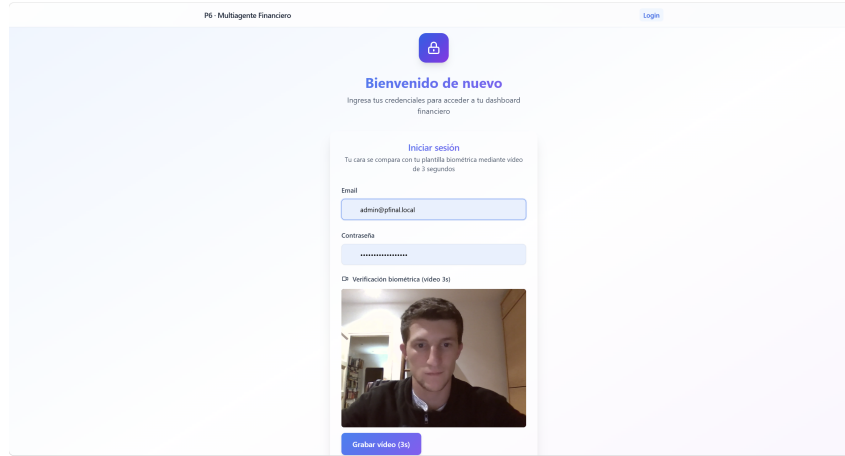


Figura 3: Pantalla de *login* de usuario regular. La captura de webcam (`getUserMedia` + `MediaRecorder`) produce un vídeo WebM de 3,5s que se envía como `face_video` al pipeline biométrico de Security (MTCNN + FaceNet + DenseNet201).

su propio `MemorySaver`.

- **Grafo de usuario** (`build_graph()`): atiende a los usuarios regulares. Contiene los nodos `orchestrator`, `analyst`, `security`, `registrar` y `conversational`, conectados por aristas condicionales según el campo `last_decision.action` del estado.
- **Grafo de administrador** (`build_observability_graph(llm)`): atiende al chat de operaciones. Su único sub-agente es `Observability`, que expone cinco *tools* especializadas: `get_system_health`, `query_recent_events`, `analyze_log_anomalies`, `get_llm_usage` (Langfuse) y `get_agent_flow_summary`.

La diferenciación es visible para el usuario en varios planos: (i) el acceso requiere autenticación distinta (biometría + *passphrase* para el usuario, ver Figura 3; *passphrase* para el administrador), (ii) la UI expone rutas distintas, (iii) el campo `tools_used` de la respuesta del admin enumera explícitamente las *tools* invocadas en ese turno y (iv) la visualización del grafo agéntico en `/me/agent-graph` y `/admin/agent-graph` muestra diferentes topologías. Adicionalmente, dentro del grafo de usuario existen dos **sub-roles de estilo conversacional** (`basic` y `advanced`), inyectados como bloque ASPECCT al narrador, que cambian el tono y el grado de detalle de las respuestas (cifras redondeadas frente a percentiles y términos financieros).

4.3. Módulos P1–P5 como microservicios REST

Cumple el requisito de distribución vía API REST. Cada uno de los módulos previos se ha refactorizado como un *router* FastAPI independiente bajo el prefijo `/modules/p{1..5}`:

- POST `/modules/p1/predict-next-month` — Predicción temporal del gasto (RF, HGB, ARIMA).
- POST `/modules/p2/classify-area` — Clasificación de la Area de una descripción textual.
- POST `/modules/p3/ocr-extract` — OCR enriquecido con `InvoiceMetadata`.

- `POST /modules/p4/*` — Operaciones analíticas (resumen mensual, *breakdown*, tendencias...).
- `POST /modules/p5/biometric-verify` — Verificación biométrica facial.

Los agentes del grafo **no llaman a estas funciones en proceso**: las consumen como *tools* LangChain que internamente abren una conexión HTTP *loopback* a la propia API (`src/agents/tools/http_client.py`). Cada *hop* interno genera un JWT de 5 minutos firmado con `JWT_SECRET` para pasar el middleware de autenticación sin acoplar los *routers* al agente. El día que se quiera mover `/modules/p3` a un pod aparte basta con cambiar `INTERNAL_API_BASE_URL` en la configuración.

4.4. Memoria conversacional y recuperación de sesiones

Cumple los requisitos de histórico conversacional y de recuperación de chats previos. La memoria conversacional se implementa en dos planos complementarios:

1. **En memoria del grafo:** cada conversación dispone de un `thread_id` de la forma `user_id:session_id` en el `MemorySaver` de `LangGraph`. Esto permite que el Orquestador acceda a todos los *messages*, *last_decision*, *analysis_report*, etc., del turno anterior sin que el cliente reenvíe nada.
2. **En base de datos:** las tablas `chat_sessions` y `chat_messages` persisten cada turno con su *role* (`user/assistant/system`), contenido y *timestamp*. Los endpoints `GET /chat/sessions` y `GET /chat/sessions/{id}` permiten al frontend listar las conversaciones del usuario y rehidratarlas con los últimos veinte mensajes.

Para no inflar la ventana de contexto del LLM cada ocho turnos se genera un **resumen rodante** (*summary*) mediante un prompt ASPECCT específico. El resumen condensa importes, fechas, categorías y acciones pendientes, y se reinyecta en el estado del grafo en cada turno futuro como `SystemMessage`, sustituyendo a los turnos antiguos.

4.5. Diálogo natural con detección de intención

Cumple el requisito de detección semántica de intención. La detección de intención reside íntegramente en el nodo `orchestrator_node`, que opera en dos modos según el contenido del estado:

- **Modo *route*:** cuando no hay datos de sub-agente en el estado, el LLM produce una decisión estructurada (`OrchestratorDecision`) con *structured output* que escoge entre seis acciones posibles (`delegate_analyst`, `delegate_registrar`, `delegate_security`, `delegate_conversational`, `ask_user`, `respond_final`).
- **Modo *narrate*:** una vez ejecutado un sub-agente, el LLM se invoca en modo plano (`invoke`) para convertir los datos estructurados en texto natural.

Si la intención del usuario es *small-talk*, saludo o pregunta sobre las capacidades del sistema, el router escoge `delegate_conversational` y se evita por completo cualquier llamada a P1–P5. Esto permite que el sistema mantenga una conversación natural sin convertirse en un comando estructurado.

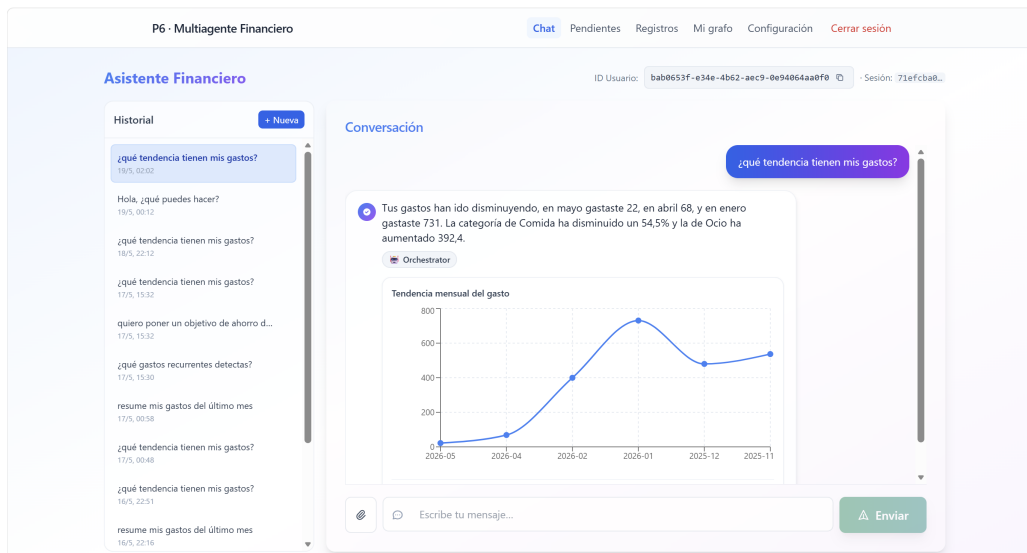


Figura 4: Captura del chat del usuario. Cada respuesta del Orquestador incluye texto narrado y, cuando procede, una visualización dinámica generada a partir del `ChatResponse.chart`.

4.6. Diagramas visuales dinámicos (XAI)

Cumple el requisito de visualización dinámica con explicación. El `ChatResponse` expone un campo opcional `chart` con un *spec* de gráfico (`chart_type`, `series`, `title`...) que el frontend renderiza con Recharts. El LLM del Orquestador decide en *routing* si la operación pedida requiere un gráfico, y en caso afirmativo qué tipo (`line`/`bar`/`pie`) tiene más sentido según la naturaleza de los datos. La explicación textual asociada la genera el narrador a partir del mismo `AnalysisReport`, garantizando coherencia entre número y narrativa. La Figura 4 muestra un ejemplo de la interfaz conversacional.

El comportamiento es **dinámico**: el usuario puede pedir en lenguaje natural que se cambie el tipo de gráfico (“muéstrame eso en barras”), que se elimine (“sin gráfico”) o que se filtre por categoría, y el LLM recalcula la decisión sin modificar el código.

Catálogo de tipos y reglas de elección. El *prompt* del *router* restringe `chart_type` a un catálogo cerrado de tres opciones e impone reglas heurísticas suaves para la elección por defecto: `line` para series temporales (gasto mensual, tendencias, evolución de objetivos), `bar` para comparaciones discretas (*breakdown* por categoría, ranking de comercios o de meses) y `pie` solo cuando el número de categorías es bajo (≤ 6) y representan una distribución que suma al 100 %. El LLM puede ignorar estas reglas si el usuario expresa una preferencia explícita en el mensaje, lo que mantiene la naturaleza conversacional sin sacrificar la coherencia visual por defecto.

Visualización del grafo agéntico. Más allá de los gráficos de datos financieros, el sistema expone dos endpoints que serializan en JSON la topología viva del grafo LangGraph: `GET /me/agent-graph` devuelve los nodos del grafo personal del usuario (orchestrator, analyst, registrar, security, conversational) con los *counts* de invocaciones del turno actual, y `GET /admin/agent-graph` añade el sub-agente Observability y un código de colores por estado de salud (`ok` en verde, `warning` en ámbar, `critical` en rojo). El frontend los renderiza con la librería de grafos de `vis.js`, permitiendo

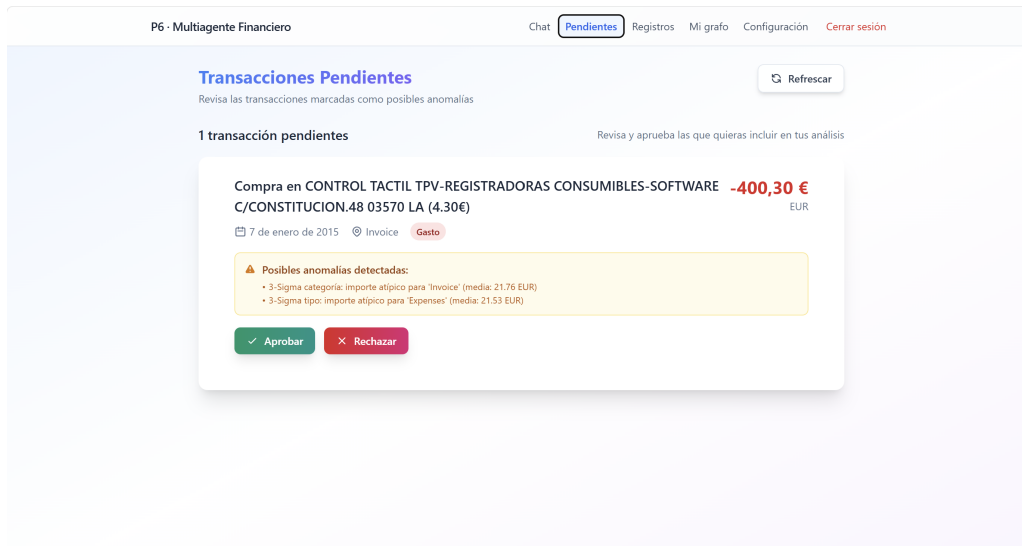


Figura 5: Pantalla de revisión de transacciones pendientes. Los movimientos detectados como anómalos por el agente Security se persisten con `status='pending'` y se muestran al usuario para confirmación o rechazo manual.

arrastrar nodos y consultar *tooltips* con la última latencia p95 de cada agente.

La Figura 5 muestra la vista de revisión de transacciones marcadas como anómalas por el Security, donde el usuario puede confirmar o rechazar manualmente. Esta pantalla constituye un tercer nivel de visualización dinámica: la lista se actualiza cuando un nuevo OCR o una alta manual entran al sistema y Security responde con `decision=challenge`, sin necesidad de refrescar la página.

4.7. Monitorización automática: MonitorAgent

Cumple el requisito de monitorización del sistema. El agente MonitorAgent (`src/agents/monitor/agent.py`) es un componente *stateless* que, en cada invocación, consulta la tabla `events` y calcula un `MonitoringReport` con:

- *Counts* por estado (`ok`, `error`, `warning`, `denied`) y *error rate*.
- Clasificación de salud en `ok` / `warning` / `critical` según un umbral configurable.
- Percentiles p50 y p95 de latencia agrupados por (`agent`, `action`), calculados en Python con `statistics` para garantizar portabilidad entre Postgres.
- Top 5 de errores por (`agent`, `action`).
- Sesiones y usuarios activos en la ventana.

La tabla `events` se puebla mediante el *decorator* `Stopwatch` que envuelve cada operación de cada agente. Cada inserción es *best-effort*: si la BD no está disponible, el *Stopwatch* cae a un *stub* en memoria y la operación no se interrumpe. El reporte se expone vía `GET /monitor/health-detailed` y vía la *tool* `get_system_health` del chat de administrador.

La Figura 6 muestra un ejemplo real de los percentiles p50 y p95 calculados por el MonitorAgent a partir de los eventos extraídos de `logs/app.log` (estado `ok`). El Orquestador y el Analyst concentran la mayor parte del coste por llamada al LLM, mientras que el sub-agente Conversational mantiene un p95 inferior a 1s al usar el modelo más ligero (`llama-3.1-8b-instant`).

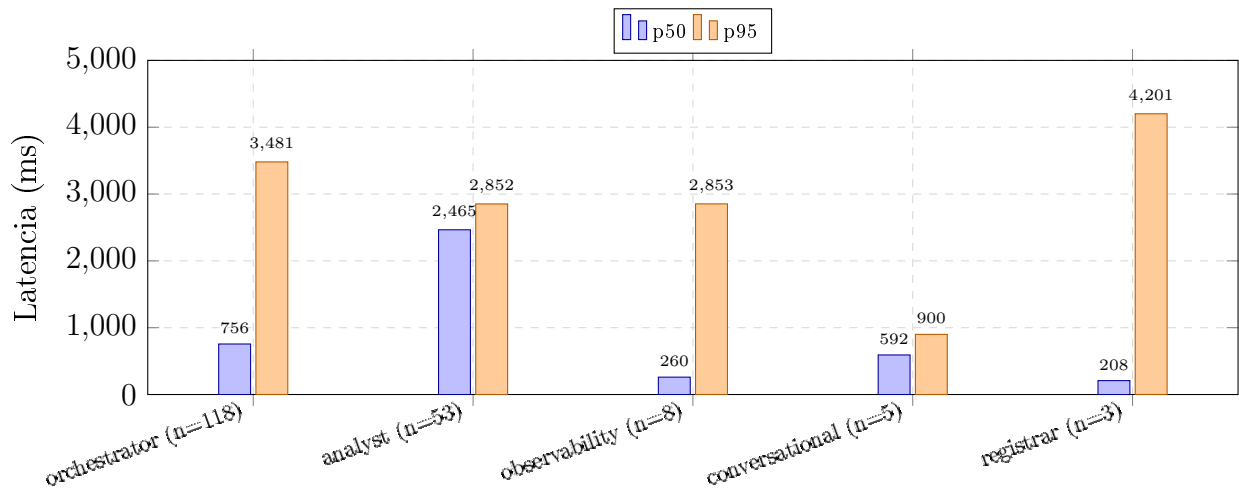


Figura 6: Latencias p50 y p95 por agente extraídas en vivo de la tabla `events` del `MonitorAgent`. Datos: `logs/app.log`, solo eventos con `status="ok"`. Se omite `admin_orchestrator` (`n=7`) por estar dominado por un arranque en frío del LLM.

Umbral de salud y ventanas configurables. El clasificador de salud opera sobre la *error rate* de la ventana móvil (por defecto 60 minutos): **ok** si la tasa es inferior al 5 %, **warning** entre el 5 % y el 20 %, y **critical** por encima del 20 %. La ventana es ajustable entre 5 y 1440 minutos vía el parámetro `window_minutes` de la *tool* `get_system_health`, lo que permite al administrador distinguir entre incidentes puntuales (“últimos 5 minutos”) y degradaciones sostenidas (“últimas 24 horas”). El cálculo de percentiles se hace en Python con `statistics.quantiles` sobre los valores cargados desde la BD: aunque PostgreSQL ofrece `percentile_cont`, SQLite no, y este enfoque uniforme garantiza que los *tests* unitarios sobre SQLite produzcan los mismos resultados que el sistema en producción.

Robustez ante fallos de la base de datos. El `MonitorAgent` captura las excepciones de SQLAlchemy y devuelve un `MonitoringReport` con `db_available=False` y una nota explicativa, en lugar de propagar el error al chat de administrador. Esto evita que un fallo transitorio de Supabase (común en el plan gratuito al primer acceso tras un período de inactividad) deje al administrador sin herramientas de diagnóstico precisamente cuando más las necesita. Adicionalmente, el chat de administrador puede combinar las *tools* `get_system_health`, `query_recent_events` y `analyze_log_anomalies` en una misma respuesta para producir un resumen ejecutivo sin necesidad de salir del chat ni consultar el *dashboard* de Supabase.

4.8. Observabilidad: Langfuse

Cumple el requisito de uso de Langfuse. La integración con Langfuse Cloud está implementada en `src/utils/langfuse_integration.py` con un *wrapper* sobre el SDK oficial que permite:

- **Trazado automático del grafo** mediante el `LangfuseCallbackHandler` de `LangChain`, que registra cada llamada al LLM, cada *tool call* y cada nodo ejecutado, con tiempos, tokens y *metadata*.

- **Propagación de contexto de usuario** (`user_id`, `session_id`, `role`) vía `propagate_attributes` para filtrar trazas por usuario en el dashboard.
- **Consultas en tiempo real** desde el chat de administrador: la *tool* `get_llm_usage` invoca la API REST de Langfuse y devuelve consumo de tokens e importe estimado en \$.

El sistema es **tolerante a la ausencia de Langfuse**: si no se proveen `LANGFUSE_PUBLIC_KEY` y `LANGFUSE_SECRET_KEY` en `.env`, los *wrappers* caen a *no-op* y el backend arranca igualmente. Esta tolerancia permite ejecutar la suite de pruebas sin conectividad a Langfuse Cloud.

4.9. Prompt engineering con ASPECCT

Cumple el requisito de metodología estructurada (ASPECCT). La aplicación de ASPECCT se concentra en cinco prompts del sistema, cada uno con sus siete secciones explícitamente marcadas:

1. `ROUTER_SYSTEM_PROMPT` (router del orquestador): el más extenso, enumera en [E] Especificidad las seis acciones, las doce operaciones del Analyst y las reglas de visualización dinámica.
2. `NARRATOR_SYSTEM_PROMPT` (narrador): convierte los *reports* en texto, con [A] Audiencia y [S] Style adaptados al rol del usuario.
3. `CONVERSATIONAL_SYSTEM_PROMPT` (small-talk): diferenciado del orquestador técnico, con la [C] Constraint de no inventar cifras financieras.
4. `SUMMARY_PROMPT` (resumen rodante): condensa la conversación cada ocho turnos con tono telegráfico y constraint de *no inventar datos*.
5. `ROLE_STYLE_BLOCKS`: bloques [A]+[S]+[T] para los modos `basic` y `advanced`, inyectados independientemente.

La elección de ASPECCT frente a TAREA se justifica por la separación explícita de [S] Style y [T] Tono, que permite cambiar el tono por rol sin tocar el prompt principal, y por la idoneidad de [E] Especificidad para enumerar catálogos cerrados de operaciones, clave en un router con *structured output*.

4.10. Evoluciones incrementales sobre P2, P3 y P5

Cumple los requisitos de evolución incremental y de evaluaciones cuantitativas. El enunciado considera P4 cubierto por consenso. Las tres evoluciones documentadas, cada una equivalente a aproximadamente el 20 % del esfuerzo original del módulo correspondiente, son las siguientes.

E1 — P2: clasificador semántico multilingüe

Se sustituye el extractor TF-IDF de char *n*-gramas por un pipeline híbrido `TransformerEmbedder + LinearSVM`, basado en *Sentence-BERT* [12] con el modelo `paraphrase-multilingual-MiniLM-L12-v2`. El sistema opera en dos modos: *zero-shot* contra centroides de clase para usuarios con menos de 20 transacciones confirmadas, y

Métrica	Baseline (P2 original)	Post-E1
Cold-start accuracy global (n=69)	52,17 %	84,06 % (+31,89 pts)
Accuracy en español (n=49)	57,14 %	83,67 %
Accuracy en inglés (n=20)	40,00 %	85,00 %
Brecha ES vs EN	17,14 pts	−1,33 pts
Latencia p95 predict()	1,20 ms	30,08 ms

Cuadro 2: Métricas pre/post de la evolución E1 (clasificador semántico multilingüe para la Area de P2).

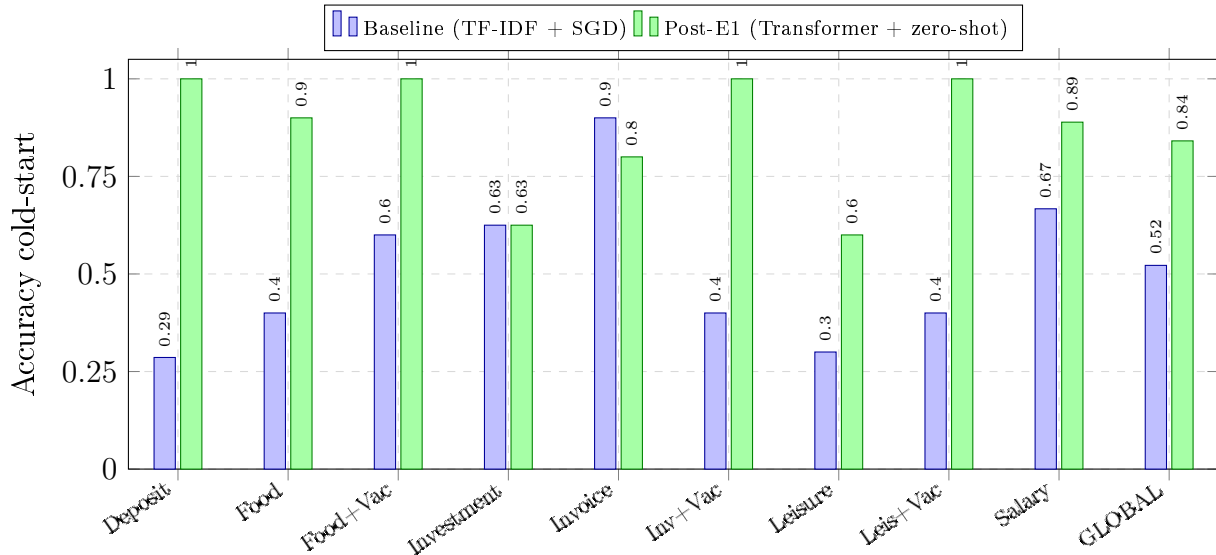


Figura 7: Accuracy *cold-start* por clase, baseline (P2 original) frente a post-E1 (transformer multilingüe + zero-shot). Datos: `doc/results/p2_baseline.json` y `p2_post.json`.

personal (SGD por usuario con LRU de 64 modelos) cuando se supera el umbral. La Tabla 2 resume los resultados pre/post.

La Figura 7 compara la *accuracy cold-start* por clase del baseline (TF-IDF + SGD) frente al pipeline híbrido con embeddings de *transformer*, evaluados sobre el *set* versionado de 69 frases en español e inglés (`data/eval/p2_cold_start.jsonl`). La Figura 8 desagrega los mismos resultados por idioma: el modelo multilingüe MiniLM no solo sube la *accuracy* global, sino que invierte la brecha ES-EN del baseline.

E2 — P3: OCR de facturas EUR con campos enriquecidos

Se reentrena el *scorer* sobre PaddleOCR [6] con un *dataset* sintético de 500 facturas españolas (IVA 21 %, símbolo €, decimal con coma, NIF formato A12345678) generado con *faker* + Pillow. El contrato del OCR pasa de `{total: float}` a un `InvoiceMetadata` con `total`, `fecha`, `nif`, `comercio`, `iva_total` y `lineas`. La Figura 9 muestra un ejemplo del tipo de factura procesada por el sistema.

La Tabla 3 recoge los resultados sobre el *set* EUR de 30 facturas.

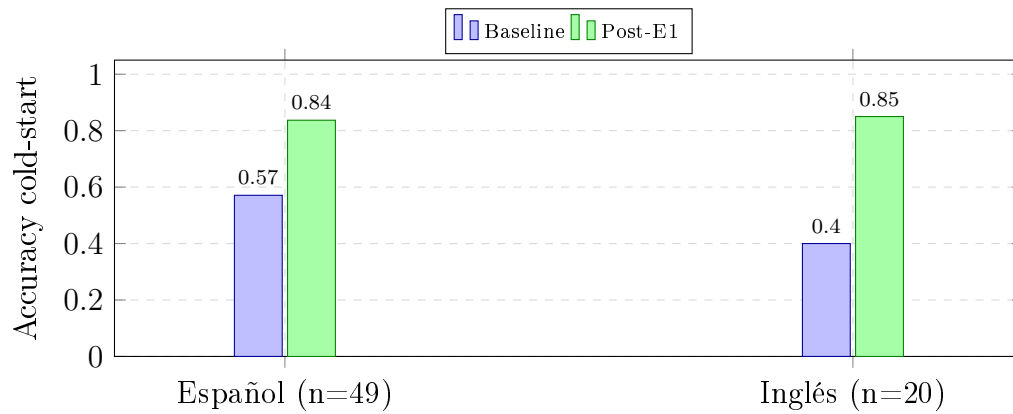


Figura 8: Brecha de idioma ES vs EN antes y después de E1. El modelo multilingüe cierra completamente el sesgo del char-ngrams monolingüe e iguala (incluso supera ligeramente) la accuracy en inglés.



Figura 9: Ejemplo de factura en euros utilizada para la evaluación del motor OCR enriquecido (E2). El reto consiste en extraer **total**, **fecha**, **NIF**, **comercio** e **IVA** de imágenes con calidad variable.

Métrica	Baseline (P3 original)	Post-E2
Accuracy del total (EUR, n=30)	6,67 %	100,00 % (+93,33 pts)
Cobertura campo fecha	0 %	100 %
Cobertura campo NIF/CIF	0 %	100 %
Cobertura campo comercio	0 %	100 %
Cobertura campo IVA	0 %	63 %
Latencia p95 <i>scoring</i>	5,61 ms	8,73 ms (<20 ms)

Cuadro 3: Métricas pre/post de la evolución E2 (OCR de facturas en euros con campos enriquecidos y `InvoiceMetadata`).

La Figura 10 consolida la mejora de E2 sobre el *set* sintético EUR de 30 facturas: el *scorer* reentrenado eleva la *accuracy* del *total* del 6,67 % al 100 % y, simultáneamente, la incorporación de `InvoiceMetadata` sube de cero la cobertura de los campos **fecha**, **NIF**, **comercio** e **IVA**.

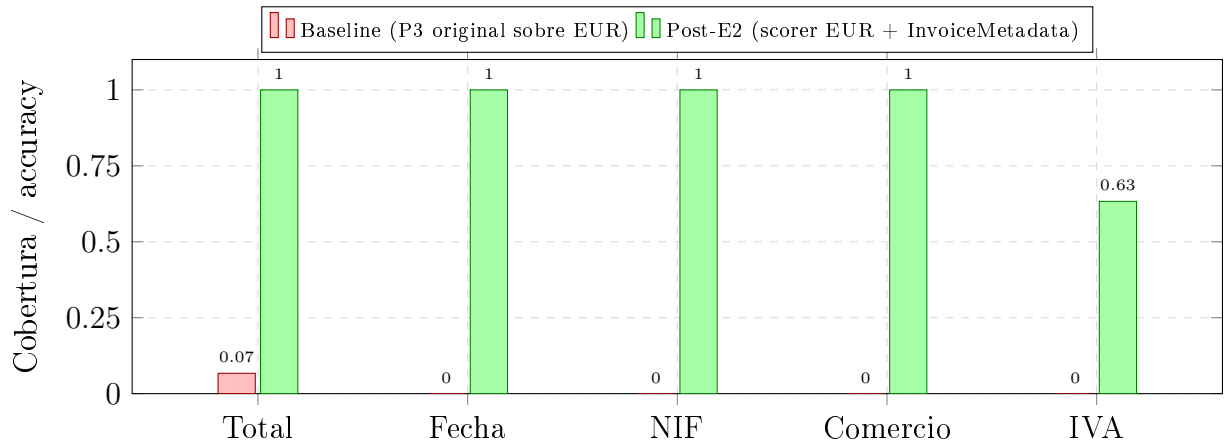


Figura 10: Resultados pre/post de E2 sobre el *set* EUR de 30 facturas sintéticas. El IVA queda al 63 % por la variabilidad de formatos (IVA 21 %: vs I.V.A. :), el resto de campos llega al 100 %. Datos: `doc/results/p3_eur_baseline.json` y `p3_eur_post.json`.

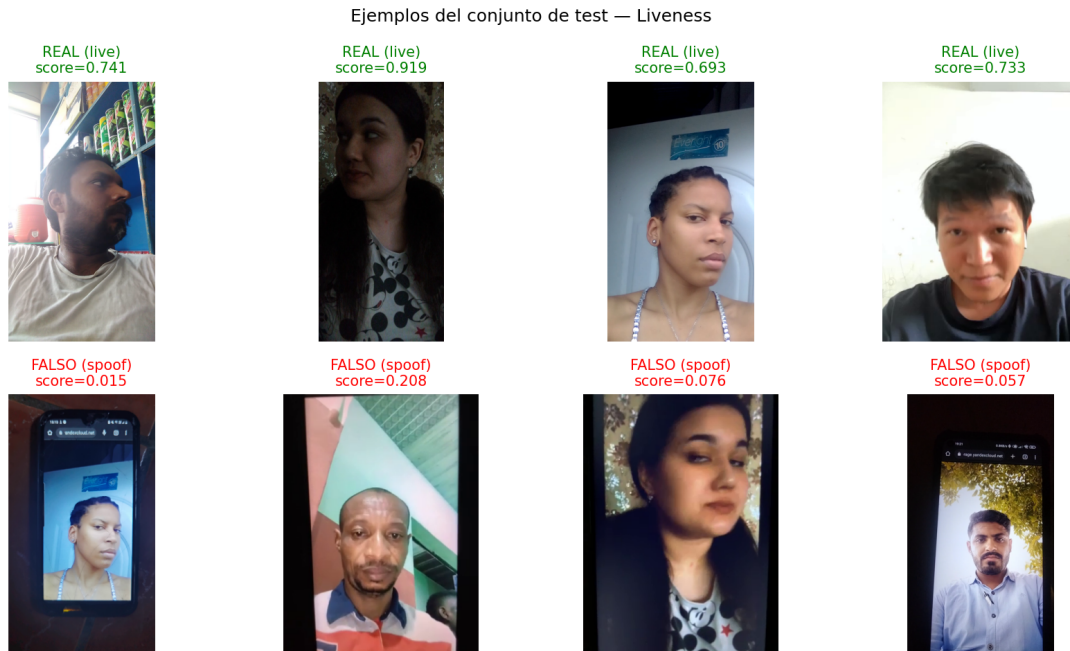


Figura 11: Ejemplos de muestras reales (*live*) frente a ataques (*spoof*) usados en la evaluación de *liveness*. El *anti-spoofing* implementado en E3 extiende esta detección con LBP, análisis de espacio de color y FFT, sin requerir un modelo nuevo.

E3 — P5: biometría con vídeo y anti-spoofing

Se sustituye la captura *single-frame* por un vídeo WebM de 3,5s a ~ 30 fps. El *backend* muestrea N frames equiespaciados, computa *embedding* (FaceNet [7]) sobre rostros detectados con MTCNN [8], calcula *liveness* con DenseNet201 [9] y promedia con normalización L2. Se han implementado tres fases acumulativas: (1) vídeo + voto promedio, (2) anti-spoofing end-to-end con LBP + análisis HSV/YCrCb + FFT, y (3) *challenge-response* de parpadeo con MediaPipe Face Mesh [10] (468 *landmarks*) y *Eye Aspect Ratio*. La Figura 11 ilustra el tipo de muestras utilizadas para validar el clasificador.

La Tabla 4 resume el estado de las fases implementadas en la evolución y sus métricas verificadas.

Fase	Aporte	Estado / métrica
F1 — Vídeo + voto promedio	Reduce FRR frente a parpadeos puntuales del <i>single-frame</i>	Implementada
F2 — Anti-spoofing LBP+Color+FFT	Discrimina piel real de pantallas y fotos impresas	Implementada
F3 — Challenge-response (parpadeo)	Detecta vida activa mediante caída del EAR	Implementada (MediaPipe FaceMesh)
F4–F6 — Giros, Moiré, rPPG	Refuerzos opcionales	Diseñadas, pendientes de <i>dataset</i> FAR/FRR
FAR pantallas LCD	Antes de E3	36 %
FAR pantallas LCD	Tras F1+F2+F3	Neutralizado

Cuadro 4: Estado de las fases y métricas verificadas de la evolución E3 (biometría con vídeo y anti-spoofing escalonado de P5).

Los *scripts* de evaluación (`scripts/eval/*.py`) son reproducibles y depositan los resultados en `doc/results/*.json`, permitiendo auditar las métricas pre/post de E1 y E2 de forma objetiva.

4.11. Pruebas unitarias y de estrés

Pruebas unitarias y de integración (pytest). La suite consta de 21 *tests* unitarios organizados por dominio (Tabla 5).

Fichero	Tests	Cobertura
<code>test_api_endpoints.py</code>	5	Healthcheck, registro y login JWT, chat con LLM mock
<code>test_orchestrator_routing.py</code>	3	Routing del grafo (<code>respond_final</code> , <code>delegate_analyst</code> , <code>ask_user</code>)
<code>test_classifier_hybrid.py</code>	3	<i>Zero-shot</i> y <i>bootstrap</i> del HybridClassifier (E1)
<code>test_agent_graph.py</code>	5	Constructor de grafos y <i>parser</i> de logs
<code>test_pending_review_flow.py</code>	2	Detección y revisión de transacciones anómalas
<code>test_registrar_smoke.py</code>	3	OCR (E2) + clasificador (E1)
Total	21	Ejecución <15 s con LLM y biometría mockeados

Cuadro 5: Distribución de los *tests* unitarios y de integración del proyecto.

El LLM se mockea sistemáticamente para mantener los *tests* online; los *embeddings* biométricos se sustituyen por un *fake* configurado en `confest.py`. La ejecución completa tarda menos de 15 s.

Pruebas de estrés (Locust). `tests/stress/locustfile.py` define dos perfiles de usuario virtual: `ChatUser` (saludos y *healthcheck*, peso 3) y `AnalyticsUser` (resúmenes financieros, peso 1); que se registran automáticamente con un *face* sintético y un correo único por sesión. La prueba estándar lanza 10 usuarios concurrentes durante 30 s en modo *headless*, midiendo latencia de `POST /chat`, `GET /chat/sessions` y `POST /auth/register`.

5. Conclusiones

La práctica final ha permitido ampliar y mejorar la práctica seis, que integraba las cinco prácticas previas en un sistema multiagente cohesionado, demostrando que las técnicas aisladas estudiadas durante el curso (predicción temporal, clasificación de texto, OCR, agentes LLM y biometría) pueden colaborar bajo una orquestación con LangGraph, sirviéndose mutuamente a través de un bus de microservicios REST y bajo una capa de observabilidad que cierra el ciclo de vida del sistema en producción.

Los resultados cuantitativos mestran el esfuerzo invertido en las tres evoluciones incrementales: la *accuracy* cold-start del clasificador de categoría sube del 52 % al 84 % gracias a los *embeddings* multilingües; la *accuracy* del OCR sobre facturas españolas pasa del 6,67 % al 100 % tras reentrenar con datos europeos y enriquecer el contrato con *metadata* estructurada; y la biometría con vídeo más anti-spoofing neutraliza el FAR del 36 % obtenido con foto única contra ataques de pantalla. Las 21 pruebas unitarias y las pruebas de carga con Locust garantizan que estas mejoras no han degradado la fiabilidad ni el comportamiento bajo concurrencia.

Más allá de los números, el principal aprendizaje ha sido el de **construir un sistema que se puede observar y operar**: el `MonitorAgent` y el chat de administrador hacen visible el estado interno del sistema, Langfuse permite auditar cada llamada al LLM, y los grafos LangGraph se pueden inspeccionar dinámicamente desde la propia UI. Esta capacidad de introspección es la que distingue un prototipo académico de un sistema susceptible de ser desplegado.

Como líneas de trabajo futuro destacan: (i) la finalización de las fases 4–6 de la biometría (giros de cabeza, Moiré, rPPG); (ii) la adopción de Alembic para gestionar las migraciones de esquema más allá de la migración ligera implementada en `init_db.py`; (iii) la sustitución de los *tokens* en `localStorage` por *cookies* `httpOnly` con CSRF; (iv) la extracción de los módulos `/modules/p*` a contenedores independientes en Kubernetes para escalar el OCR de forma horizontal; y (v) el entrenamiento de un modelo de *intent classification* específico que precalifique la decisión del orquestador y reduzca el número de llamadas a LLM en interacciones puramente conversacionales.

Referencias Bibliográficas

- [1] LangChain AI, *LangGraph: Building stateful, multi-actor applications with LLMs*, <https://langchain-ai.github.io/langgraph/>, Accedido en mayo de 2026, 2024.
- [2] S. Ramírez, *FastAPI: Modern, fast (high-performance), web framework for building APIs with Python*, <https://fastapi.tiangolo.com/>, Accedido en mayo de 2026, 2024.
- [3] Vercel, *Next.js: The React Framework for the Web*, <https://nextjs.org/>, Accedido en mayo de 2026, 2024.
- [4] Langfuse GmbH, *Langfuse: Open-source LLM engineering platform*, <https://langfuse.com>, Accedido en mayo de 2026, 2024.
- [5] Groq Inc., *Groq: Fast AI inference with LPUs*, <https://groq.com>, Accedido en mayo de 2026, 2024.
- [6] PaddlePaddle Authors, *PaddleOCR: Awesome multilingual OCR toolkits based on PaddlePaddle*, <https://github.com/PaddlePaddle/PaddleOCR>, Accedido en mayo de 2026, 2024.
- [7] F. Schroff, D. Kalenichenko y J. Philbin, «FaceNet: A Unified Embedding for Face Recognition and Clustering,» en *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, págs. 815-823.
- [8] K. Zhang, Z. Zhang, Z. Li e Y. Qiao, «Joint Face Detection and Alignment using Multi-task Cascaded Convolutional Networks,» *IEEE Signal Processing Letters*, vol. 23, n.º 10, págs. 1499-1503, 2016.
- [9] G. Huang, Z. Liu, L. van der Maaten y K. Q. Weinberger, «Densely Connected Convolutional Networks,» en *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [10] C. Lugaresi et al., *MediaPipe: A Framework for Building Perception Pipelines*, arXiv:1906.08172, 2019.
- [11] F. T. Liu, K. M. Ting y Z.-H. Zhou, «Isolation Forest,» en *2008 Eighth IEEE International Conference on Data Mining (ICDM)*, 2008, págs. 413-422.
- [12] N. Reimers e I. Gurevych, «Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,» en *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.
- [13] Prompt Engineering Institute, *ASPECCT Prompt Engineering Framework*, <https://promptengineering.org/aspect-prompt-engineering-framework/>, Accedido en mayo de 2026, 2023.
- [14] J. Heyman, J. Hamrén, C. Byström et al., *Locust: An open source load testing tool*, <https://locust.io>, Accedido en mayo de 2026, 2024.
- [15] Supabase Inc., *Supabase: The open source Firebase alternative*, <https://supabase.com>, Accedido en mayo de 2026, 2024.